

HW 6: Auctions & Strategic Network Formation

Assigned: 02/23/25

Due: 03/02/25 at 11:59 pm

Collaboration is allowed for all problems and at the top of your submission, please list all the people with whom you discussed. Also, if you used AI to help you, please write a couple sentences on how you used it. Crediting help from other classmates or AI will not take away any credit from you. The details of the collaboration and AI use policies for this course are available in the Resources tab on Piazza.

*You must turn in your homework electronically via Gradescope. **Be sure to submit your homework as a single file.***

Start early and come to office hours with your questions! We also encourage you to post your questions on Piazza, as well as answer the questions asked by others on Piazza.

1 Coding + Data Analysis [70 points]**1.1 Strategic Network Formation [35 points]**

We've considered routing games, where on a fixed network, agents can choose to update their route until a Nash Equilibrium is reached. We'll now consider the case where the agents are the nodes of a graph, and for their action, they may decide to destroy, create, or move the target of an edge adjacent to them. The agents/nodes decide which edges to create, destroy, or modify in order to maximize their utility, which is a combination of the cost to maintain an edge and the value of the edge in terms of the information it allows them to obtain/spread.

We'll first work with a linear model. Here, a node receives a benefit for every other node it is connected to, regardless of its distance from it. Formally, the payoff, $\pi_i(G)$ agent i receives from network G with n agents is

$$\pi_i(G) = -c \cdot |N_i| + \sum_{\substack{j=1 \\ j \neq i}}^n \mathbb{I}_{ij}(G)$$

where $\mathbb{I}_{ij}(G)$ is an indicator function which is 1 if there exists a path from node i to node j in G , $c > 0$ is the cost of maintaining a connection, and $|N_i|$ is the size of the set of neighbors of agent i (for a directed graph, this would be the number of outgoing edges node i has).

We will simulate what networks form in different regimes of c , and hypothesize whether they are efficient. A network is considered efficient if it maximizes the *total payoff* of a network, $\Pi(G)$, among all networks G with n nodes, where $\Pi(G) = \sum_{i=1}^n \pi_i(G)$. The simulation will work as follows:

- Initialize a graph on nine nodes. You should consider initializing from a few starting graphs, such as the empty graph, the star graph, the complete graph, and a cycle.
- At timestep t , node $t \bmod n$ takes its turn. They should consider all ways they could add, remove, or swap an edge adjacent to them to another target. They should choose an action which maximizes their payoff, or take no action if no action can improve their current payoff.
- Continue for 500 iterations, or until an equilibrium is reached

Your task:

1. Analyze the results for an undirected graph in the regimes of $c < 1$, $c > n - 1$, and $1 < c < n - 1$. For each regime, does there appear to be a unique equilibrium? If there are equilibria, are they efficient? Why do you think you get the behavior observed in each regime?
2. Run the simulation and answer these questions for a directed graph, in the same regimes of c .
3. We now introduce another model, called the connections model. In the connections model, a node's payoff decays as the distance from a target node increases. More precisely, the payoff, $\pi_i(G)$ agent i receives from network G with n agents is

$$\pi_i(G) = -c \cdot |N_i| + \sum_{\substack{j=1 \\ j \neq i}}^n \delta^{d_{ij}(G)}$$

where $d_{ij}(G)$ is the number of edges in the shortest path between agent i and agent j , $c > 0$ is the cost of forming a connection, $|N_i|$ is the size of the set of neighbors of agent i , and $\delta \in (0, 1)$ is a constant. Note that we set $d_{ij}(G) = \infty$ when there is no path between i and j in G .

Repeat the simulation on an undirected graph, using the connections model as the nodes' utility function. For the connections model, consider the regimes $c < \delta - \delta^2$, $\delta - \delta^2 < c < \delta + (n/2 - 1)\delta^2$, and $c > \delta + (n/2 - 1)\delta^2$ and answer the same questions.

1.2 Escaping the Cursed Maze [35 points]

In this problem, we will introduce some ideas from learning with games.

You and your friend were dropped into a cursed maze. Every room in this maze has four doors which appear to go North, South, East, and West. The maze is seemingly infinite, but some rooms are filled with deadly snakes, while other rooms have an opening in the ceiling which would allow you to escape. Since you were recently gifted 999 lives from the Cat Queen, you offer to explore the maze while your friend waits in safety. A snake room will cause you to respawn at the room with your friend, as will exiting and re-entering the maze at an escape room. There is another cursed feature of the maze, which is that every time you enter a standard room, there is a 20% chance that the maze floor turns into a conveyor belt which pushes you into one of the adjacent rooms against your will (each adjacent room with equal probability). Thus, it is not enough to avoid intentionally entering snake rooms, but also important to avoid routes near snake rooms in the event this happens.

1. Explain why DFS or BFS won't work here.

Luckily, you have an idea for another approach. For each room you explore, you will assign a reward. Snake rooms will have a fixed negative value, and escape rooms will have a fixed positive value, which will not change. Plain rooms will be given a zero value when they are first discovered, but you will update this value as you explore. It turns out the maze is on a torus, which means you can model it as a 10x10 grid, such that if you would move right off the grid, you end up in the same row on the left side of the grid. Similar logic works for moving off the grid in the other directions. You have a fantastic memory, so remembering this array will not be a problem, although you cannot remember anything more than this, the current room you are in, and which room you just came from. When you move from room i to i' , the value of i will be influenced by the value of i' according to

$$V(i) = (1 - \alpha)V(i) + \alpha V(i')$$

α is the learning rate, which determines how strongly you want rewards and punishments to propagate.

There's also the matter of which direction you try to explore. You decide to take an ε greedy approach, which means with probability $1 - \varepsilon$, you go through whichever door has the best value in the next room, based on your current valuations (you can decide what value to assign to unexplored rooms), and with probability ε , you choose your direction randomly.

After you've spent sufficient time learning the maze, you will lead your friend through the maze by always choosing the door with highest value on the other side. Good luck!

2. Learn valuations by exploring until you reach a terminal room (snakes or exit) over the course of 998 or less iterations, using functions from maze.py to query the maze.
3. Parameters you can consider tuning are α , ε , the reward assigned to an unexplored room, and the value of snake/trap rewards. You can change your ε over the course of your iterations as well.
4. Show that your strategy allows you to successfully escape with high probability. That is, if you run 1000 test iterations, where at each step you attempt to enter the adjacent room with the highest valuation (i.e. exploring with ε of 0), you successfully exit the maze in all 1000 of 1000 iterations. Also include your final valuations for each square of the grid that gave rise to this navigation.

2 Theory [30 points]

2.1 Routing games with tolls [10 points]

In our discussion of routing games in class, we talked about using tolls to induce independent selfish players to behave in a manner that is optimal for the system. To this end, we discussed the principle of *marginal cost pricing*. In this exercise, you'll prove that marginal cost pricing really works.

Consider the model of routing games described in class. Assume that each user i has traffic $r_i = 1$. We add a 'toll' to the original link cost functions $c_e(\cdot)$, setting the new cost function to be

$$c_e^*(x) = c_e(x) + (x - 1)(c_e(x) - c_e(x - 1))$$

Notice that the 'toll' on link e captures the net latency that each player using the link adds to the other players using it. Prove that any optimal strategy for our original game is a Nash equilibrium for this new game (with link cost functions $c_e^*(\cdot)$). Recall that the objective function in the original game is $\sum_{i \in N} \sum_{e \in E_i} c_e(f_e) = \sum_{e \in E} f_e c_e(f_e)$ where f_e is the flow on e , N is the set of players and E_i contains the edges on the path selected by player i .

Hint: Think about constructing a new potential function with the new cost.

2.2 Alternating offers [5 points]

Eric has made the following deal with Anagha and Albert: He gives them \$3000 to split. Anagha gets to make an initial offer. Albert can then either accept Anagha's offer, or reject it and propose an offer of his own. Anagha hears Albert's offer and can similarly either reject it or propose a final counter offer of her own. For simplicity, assume all offers must be non-negative integer values. Additionally, knowing that Anagha and Albert are really slow at making decisions, Eric decides to reduce the total amount of money by \$1000 every time an offer is rejected.

To give you a feel for this game, the following is a possible (although likely irrational) sequence:

- Anagha offers Albert \$1000, keeping \$2000 for herself.
- Albert rejects, offering Anagha \$0 and keeping \$2000 for himself.
- Anagha rejects, offering Albert \$500 and keeping \$500. Albert accepts.

Your task:

- (a) Assuming both of them are completely rational and want to make as much money as possible for themselves, how much should Anagha offer Albert?
- (b) Suppose Albert suddenly decides he hates Anagha. He now only accepts if he gets more than Anagha, and only cares about maximizing how much more money he gets relative (in terms of absolute difference) to his former friend. Furthermore, Albert is now spiteful: if he is made indifferent by an offer from Anagha, he will reject it, favoring the alternative in which Anagha gets less money. How much should Anagha offer Albert now?

2.3 Infinite Equilibria [5 points]

After learning about game theory and the case of infinite equilibrium in a 2×2 game, Ann makes the following statement:

"The reason that there exists an infinite number of (mixed) equilibria is that one player is indifferent between some actions. Hence, if each player's payoff is different in every cell, there should always be finite number of equilibrium."

Formally, in a two-person, $m \times m$ game, denote the set of actions of row player by $\{a_1, \dots, a_m\}$ and the set of actions of column player by $\{b_1, \dots, b_m\}$. Suppose for $i = a, b$ we have $u_i(a_j, b_k) \neq u_i(a_{j'}, b_{k'})$ for with any (j, k) and (j', k') such that $(j, k) \neq (j', k')$ (for example $u_i(a_1, b_1) \neq u_i(a_1, b_2)$, $u_i(a_1, b_1) \neq u_i(a_2, b_1)$). Then, Ann's conjecture is that there will be finite number of equilibrium.

Your task:

State whether or not you agree with Ann. If yes, prove her statement formally. If no, give a counter example and provide the maximum m such that her statement hold for $m \times m$ matrix, then prove that m is the upper bound. (Hint: Ann is wrong)

2.4 PoA analysis for Generalized Second Price Auction [10 points]

Recall the ad auction model that was introduced in lecture. We consider an auction with n advertisers and n slots. The game here is between the n advertisers who each have their own valuation v_i for a click. The strategy for each advertiser is a bid $b_i \in [0, \infty)$. There are n slots and based on the bids, we decide where to allocate each advertiser. In the simplest model, the k -th slot contains α_k clicks, where $\alpha_1 \geq \alpha_2 \geq \dots \geq \alpha_n \geq 0$. The game proceeds as follows:

- Each advertiser submits a single bid $b_i \geq 0$, which is his declared value for a click. Let $b = (b_1, b_2, \dots, b_n)$ denote the vector of all bids.
- The advertisers are sorted by their bids (ties are broken arbitrarily). Call $\pi_k(b)$ the advertiser with the k -th highest bid in b . For example, $\pi_1(b)$ would denote the advertiser with the highest bid, and $\pi_n(b)$ would denote the advertiser with the lowest bid.
- Advertiser $\pi_k(b)$ is placed on slot k and therefore receives α_k clicks.
- For each click, advertiser $\pi_k(b)$ pays $b_{\pi_{k+1}(b)}$, which is the next highest bid (assume that the lowest bidding advertiser pays nothing, essentially getting the last slot for free).

The vector $\pi(b)$ is a permutation that indicates to which slot each player is assigned – it is completely determined by the vector of bids, b . The utility of advertiser i when his ad is occupying slot j is given by

$$U_i(b) = \alpha_j(v_i - b_{\pi_{j+1}(b)}).$$

The “social welfare” of this game is the total value the bidders get from playing it, which is

$$W(b) = \sum_j \alpha_j v_{\pi_j(b)}.$$

Price of anarchy: We will measure the efficiency of this auction by the “price of anarchy”, which, for this game, is defined as the ratio of the optimal social welfare to the social welfare of the worst Nash equilibrium of the game. Formally, let b^* denote the vector of bids that maximizes the social welfare $W(b)$, and let NE denote the set of all Nash equilibrium bid vectors. Then,

$$\text{PoA} = \frac{W(b^*)}{\min_{b \in NE} W(b)}.$$

To keep things simple, in this problem, we make the following assumptions.

- We consider only *pure* Nash equilibria in the analysis of the price of anarchy. This is often called the *pure* price of anarchy or PPoA. There are results that bound the general price of anarchy too.
- We consider the complete information setting, where each bidder knows the true valuations of all the other bidders. There are results that apply to the more realistic Bayesian setting where bidders don’t know each other’s values, but have beliefs about them.

Your task:

- (a) **[3 points]** Show that the (pure) price of anarchy can be arbitrarily large. Specifically, consider the case when $n = 2$. For this case, given any $r > 1$, construct an example for which the (pure) price of anarchy is r . For a hint on how to approach this, read more of the question!
- (b) It turns out that ‘bad’ equilibria like the ones in (a) are unnatural, in the sense that the equilibrium includes (weakly) dominated strategies, which could expose the advertiser to a risk of obtaining a negative utility. Let us define a bid b_i for an advertiser to be *conservative*, if $b_i \leq v_i$. The strategy space for a *conservative* advertiser i is therefore, $[0, v_i]$.
 - (i) **[2 points]** Identify a dominated strategy employed in the example you provided for (a), and explain the risk involved for the corresponding advertiser. Recall that a strategy b_i is dominated if there is some b'_i such that $U_i(b_i, b_{-i}) \leq U_i(b'_i, b_{-i})$ for all b_{-i} and for at least one value of b_{-i} , the inequality is strict.
 - (ii) **[2 points]** Show that non-conservative bids are always dominated strategies for $n \geq 2$.
 - (iii) **[3 points]** When $n = 2$, show that if both advertisers are conservative, then the PPoA is upper bounded by 1.25. Construct an instance of the auction by specifying $\alpha_1, \alpha_2, v_1, v_2$ and a pure Nash equilibrium to show that this upper bound can be attained.

Hint: You can scale α and v to simplify your argument without loss of generality by assuming $\alpha_1 = 1, \alpha_2 = r \leq 1, \alpha_1 v_1 + \alpha_2 v_2 = 1$ and $v_1 > v_2$.